Architecture Document: StudyResourceApp

1. Application Architecture

Selected Architecture: Monolithic with Serverless Capabilities

For the **StudyResourceApp** project, we have selected a **Monolithic Architecture** leveraging the **Next.js framework**. Due to how Next.js operates on modern cloud platforms, this application acts as a monolith during local development while functioning with **Serverless** characteristics across its API Routes when deployed in production.

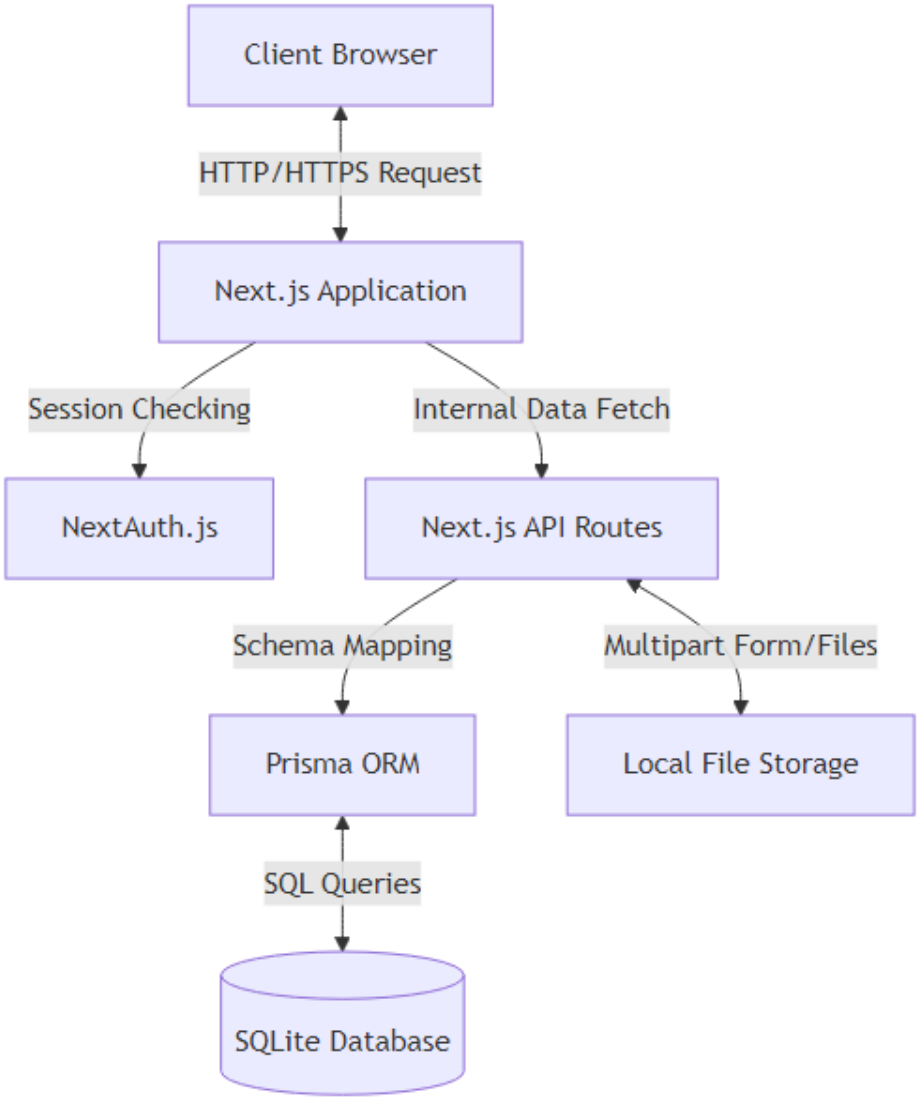
Detailed Explanation: In this architecture, both the frontend UI interface (React Components) and the backend (Next.js API Routes) share the same underlying repository and are orchestrated as a single cohesive unit.

This approach was selected for the following reasons:

- **Simplicity & Development Speed:** Having a single codebase drastically simplifies development. There is no need to orchestrate multiple distributed services.
- **Server-Side Advantages:** We seamlessly integrate server-rendered React components with direct backend data integrations, ensuring lightning-fast initial load times for students querying study resources.
- **Scalability:** By leveraging Next.js routing, individual API endpoints execute as isolated "serverless functions." This effectively eliminates maintaining persistent servers while ensuring the codebase inherits the scalability advantages of Serverless environments implicitly.

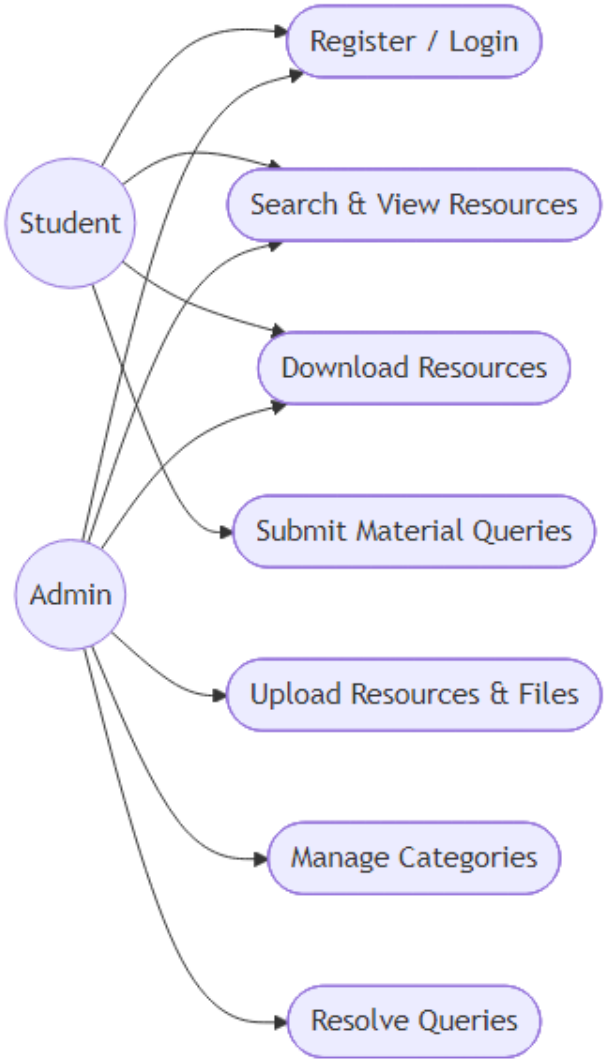
Component / Architecture Diagram

The following diagram illustrates the flow of data from the end-user out toward our database and storage.



Use Case Diagram

This identifies the varying authorization boundaries between our two distinct user roles.

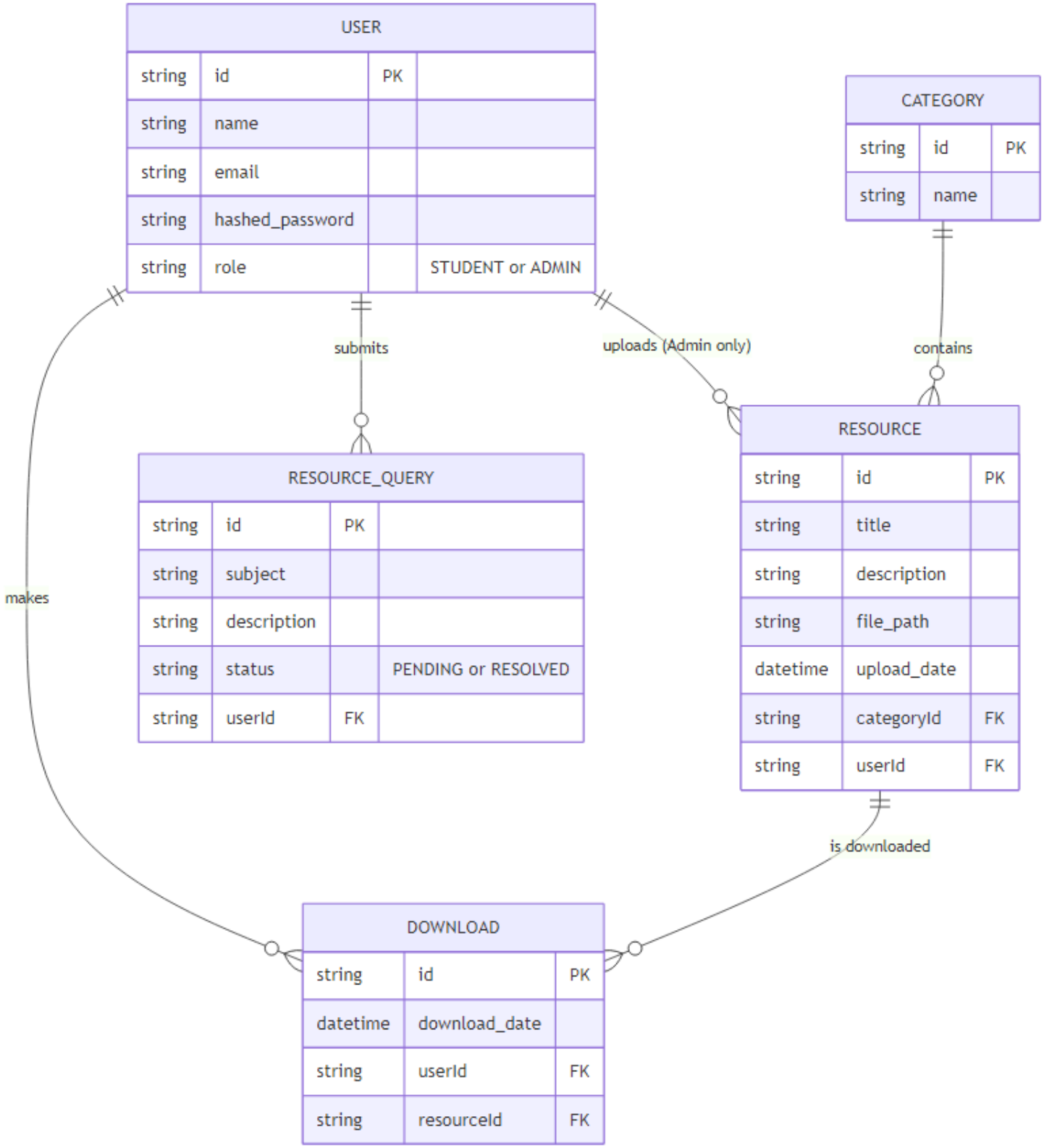


---

2. Database

Entity Relationship (ER) Diagram

The relations between resources, the users uploading them, and the users downloading them.



Schema Design

Our project utilizes a relational database structure dynamically mapped and enforced by Prisma ORM.

- **User**: Stores authenticated user information. The application manages Authorization workflows directly via the `role` enum (`STUDENT` or `ADMIN`).
- **Category**: Tracks discrete university subjects (e.g., Mathematics, Computing). It ensures resources are neatly cataloged.

- **Resource**: The centralized media entity representing an uploaded file. It natively tracks who uploaded it and links back to its file directory path.
- **Download**: Forms a many-to-many relationship tracking link between Students and the Resources they interact with.
- **ResourceQuery**: Operates as a dynamic queue. Enables Students to formally request missing academic topics, granting Admins a workflow to track, upload, and update the status to "RESOLVED".

---

3. Data Exchange Contract

Frequency of Data Exchanges

- **On-Demand Initial Loading:** Data is dynamically fetched using asynchronous HTTP `GET` requests strictly when a user renders a page on their Dashboard. We optimize this frequency to minimize stale data.
- **Event-Triggered Interactions:** Exchanges transpire upon explicit user actions triggering updates (`POST`, `PATCH`, `DELETE` requests) (e.g., clicking to submit a new file query, or updating the resolved status).

Data Sets

The application relies strictly on JSON schemas correlating to Prisma entities for frontend-backend parsing. Prominent data sets exchanged include:

1. **Resource Payload:** `{ id, title, description, file_path, category, uploaded_by }`
2. **Query Payload:** `{ id, subject, description, status, userId, createdAt }`
3. **Category Definitions:** `{ id, name }`
4. **Form Data (Multipart/Form-Data):** Standard JSON is abandoned strictly for resource uploads in favor of Multipart-Data to securely transport binary files (PDFs, DOCX).

Mode of Exchanges

- **REST-ful APIs:** Standard stateless API communications span standard HTTP methods executing natively inside Next.js API Routes.
- **File System Streams:** Binary files route from the UI to API securely where they get hashed with unique identifier tags and resolved to our internal `/public` server file system.
- **Prisma Client Interactions:** The backend communicates to the actual Database sets through asynchronous Prisma promises transcompiled dynamically into Safe-SQL queries.